JPMC DSA 100 — Questions + Links + Java 21 Solutions

JPMorgan Chase India • SWE / SEP / SDE preparation

Accuracy: JPMC does not publish an official 100-question list. ■ marks a candidate-reported/custom JPMC pattern; the rest are high-value preparation questions for the same concepts. Custom reported questions are not given invented LeetCode links.

How to use: Attempt first, then study the approach and Java 21 solution. Practice explaining brute force → optimization → complexity → edge cases.

1. Two Sum

Topic: Array / HashMap **Difficulty:** Easy **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/two-sum/

Solution idea: HashMap: store value -> index; for each x, look for target-x.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
Map<Integer,Integer> m=new HashMap<>();
for(int i=0;i<nums.length;i++){int need=target-nums[i];
if(m.containsKey(need)) return new int[]{m.get(need),i}; m.put(nums[i],i);}
return new int[0];
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

2. Maximum Subarray

Topic: Array / Kadane **Difficulty:** Easy **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/maximum-subarray/

Solution idea: Kadane: keep best subarray ending at current index.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
int cur=nums[0],ans=nums[0];
for(int i=1;i<nums.length;i++){cur=Math.max(nums[i],cur+nums[i]);ans=Math.max(ans,cur);}
return ans;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

3. Best Time to Buy and Sell Stock

Topic: Array / Greedy **Difficulty:** Easy **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/best-time-to-buy-and-sell-stock/

Solution idea: Track minimum price so far and best profit.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
int min=Integer.MAX_VALUE,ans=0;
for(int p:prices){min=Math.min(min,p);ans=Math.max(ans,p-min);} return ans;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

4. Contains Duplicate

Topic: HashSet **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/contains-duplicate/

Solution idea: Insert each value into a set; duplicate means already present.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
Set<Integer> s=new HashSet<>();
for(int x:nums) if(!s.add(x)) return true; return false;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

5. Majority Element

Topic: Boyer-Moore **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/majority-element/

Solution idea: Boyer-Moore voting maintains candidate and counter.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

6. Move Zeroes

Topic: Two Pointers **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/move-zeroes/

Solution idea: Compact non-zero values at the front, then fill remaining positions with zero.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

7. Product of Array Except Self

Topic: Prefix/Suffix **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/product-of-array-except-self/

Solution idea: Build prefix products in result, then multiply by suffix product.

Complexity: Time O(n) | Space O(1) extra

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

8. Subarray Sum Equals K

Topic: Prefix Sum + HashMap **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/subarray-sum-equals-k/

Solution idea: If prefix-k has appeared c times, it creates c subarrays ending here.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
Map<Integer,Integer> m=new HashMap<>(); m.put(0,1);
int sum=0,ans=0;
for(int x:nums){sum+=x;ans+=m.getOrDefault(sum-k,0);m.merge(sum,1,Integer::sum);}
return ans;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

9. Merge Intervals

Topic: Intervals **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/merge-intervals/

Solution idea: Sort by start; merge whenever next.start <= current.end.

Complexity: Time O(n log n) | Space O(n)

Java 21 solution:

```
Arrays.sort(intervals,Comparator.comparingInt(a->a[0]));
List<int[]> r=new ArrayList<>();
for(int[] x:intervals){
if(r.isEmpty()||r.get(r.size()-1)[1]<x[0]) r.add(x.clone());
else r.get(r.size()-1)[1]=Math.max(r.get(r.size()-1)[1],x[1]);
} return r.toArray(new int[0][]);
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

10. Insert Interval

Topic: Intervals **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/insert-interval/

Solution idea: Add intervals ending before new one, merge overlaps, append rest.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

11. Maximum Product Subarray

Topic: DP / Array **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/maximum-product-subarray/

Solution idea: Track current maximum and minimum because a negative can swap them.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

12. Find the Duplicate Number

Topic: Fast/Slow Pointers **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/find-the-duplicate-number/

Solution idea: Treat values as next pointers; Floyd cycle detection finds duplicate.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

13. Longest Consecutive Sequence

Topic: HashSet **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/longest-consecutive-sequence/

Solution idea: Start a sequence only when x-1 is absent; extend forward.

Complexity: Time O(n) avg. | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

14. Sort Colors

Topic: Dutch National Flag **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/sort-colors/

Solution idea: Maintain low, mid and high regions for 0,1,2.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

15. Circular Dial Optimization ■

Topic: Custom JPMC-reported pattern **Difficulty:** Medium **Priority:** ★★★★★

Problem link: Custom/reported pattern — no single canonical problem page.

Solution idea: Model each dial's circular distance to the required character; for multiple dials, use DP over dial positions. Exact wording varies by report.

Complexity: Time Depends | Space Depends

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

16. Valid Anagram

Topic: HashMap / Counting **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/valid-anagram/

Solution idea: Count characters in one string and subtract using the other.

Complexity: Time O(n) | Space O(1) alphabet

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

17. Group Anagrams

Topic: HashMap / String **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/group-anagrams/

Solution idea: Use sorted string or frequency signature as map key.

Complexity: Time O(n k log k) | Space O(nk)

Java 21 solution:

```
Map<String,List<String>> m=new HashMap<>();
for(String s:strs){char[] c=s.toCharArray();Arrays.sort(c);
m.computeIfAbsent(new String(c),k->new ArrayList<>()).add(s);}
return new ArrayList<>(m.values());
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

18. Top K Frequent Elements

Topic: HashMap + Heap **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/top-k-frequent-elements/

Solution idea: Count frequencies; maintain a size-k min-heap.

Complexity: Time O(n log k) | Space O(n)

Java 21 solution:

```
Map<Integer,Integer> f=new HashMap<>();
for(int x:nums) f.merge(x,1,Integer::sum);
PriorityQueue<int[]> pq=new PriorityQueue<>(Comparator.comparingInt(a->a[1]));
for(var e:f.entrySet()){pq.offer(new int[]{e.getKey(),e.getValue()});if(pq.size()>k)pq.poll();}
int[] r=new int[k];for(int i=0;i<k;i++)r[i]=pq.poll()[0];return r;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

19. First Unique Character in a String

Topic: Counting **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/first-unique-character-in-a-string/

Solution idea: Count then scan for first frequency one.

Complexity: Time O(n) | Space O(1) alphabet

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

20. Isomorphic Strings

Topic: HashMap **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/isomorphic-strings/

Solution idea: Maintain both direction mappings to enforce one-to-one correspondence.

Complexity: Time O(n) | Space O(1) alphabet

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

21. Happy Number

Topic: HashSet / Math **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/happy-number/

Solution idea: Repeated digit-square sums either reach 1 or cycle; use a set.

Complexity: Time O(log n) per iteration | Space O(log n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

22. Contains Duplicate II

Topic: HashSet / Sliding Window **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/contains-duplicate-ii/

Solution idea: Keep only the last k indices in a set.

Complexity: Time O(n) | Space O(k)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

23. Valid Parentheses / Balanced Brackets

Topic: Stack **Difficulty:** Easy **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/valid-parentheses/

Solution idea: Push opening brackets; every closing bracket must match the stack top.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
Deque<Character> st=new ArrayDeque<>();
for(char c:s.toCharArray()){
if(c=='('||c=='['||c=='{') st.push(c);
else {if(st.isEmpty()) return false; char o=st.pop();
if((c==')'&&o!='(')||(c==']'&&o!='[')||(c=='}'&&o!='{')) return false;}}
return st.isEmpty();
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

24. Longest Substring Without Repeating Characters

Topic: Sliding Window **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/longest-substring-without-repeating-characters/

Solution idea: Move left past the previous occurrence of the current character.

Complexity: Time O(n) | Space O(min(n,alphabet))

Java 21 solution:

```
int[] last=new int[128]; Arrays.fill(last,-1);
int l=0,ans=0;
for(int r=0;r<s.length();r++){l=Math.max(l,last[s.charAt(r)]+1);
last[s.charAt(r)]=r; ans=Math.max(ans,r-l+1);} return ans;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

25. Count Substrings With Non-Repeating Characters ■

Topic: Sliding Window **Difficulty:** Medium **Priority:** ★★★★★

Problem link: Custom/reported pattern — no single canonical problem page.

Solution idea: Reported JPMC-style pattern. Maintain a window with unique characters; count valid windows ending at each right index. Exact constraints vary by interview.

Complexity: Time O(n) | Space O(alphabet)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

26. Valid Palindrome

Topic: Two Pointers **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/valid-palindrome/

Solution idea: Skip non-alphanumeric characters and compare lowercase ends.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

27. Longest Common Prefix

Topic: String **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/longest-common-prefix/

Solution idea: Shrink prefix until every string starts with it.

Complexity: Time O(total chars) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

28. Reverse Words in a String

Topic: String **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/reverse-words-in-a-string/

Solution idea: Trim/split or scan from right to left and rebuild.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

29. Longest Palindromic Substring

Topic: Expand Around Center **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/longest-palindromic-substring/

Solution idea: Expand from each character and each gap.

Complexity: Time O(n²) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

30. String Compression

Topic: Two Pointers **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/string-compression/

Solution idea: Read runs and write character plus count in place.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

31. Permutation in String

Topic: Sliding Window **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/permutation-in-string/

Solution idea: Compare 26-character frequency windows.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

32. Minimum Window Substring

Topic: Sliding Window **Difficulty:** Hard **Priority:** ★★★

Problem link: https://leetcode.com/problems/minimum-window-substring/

Solution idea: Expand until all required counts are met, then shrink from left.

Complexity: Time O(n) | Space O(1) alphabet

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

33. Maximum Sum Subarray of Size K

Topic: Sliding Window **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://www.geeksforgeeks.org/window-sliding-technique/

Solution idea: Maintain sum of a fixed-size window and update maximum.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

34. Minimum Size Subarray Sum

Topic: Sliding Window **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/minimum-size-subarray-sum/

Solution idea: Expand right until sum >= target, then shrink left.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

35. Longest Repeating Character Replacement

Topic: Sliding Window **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/longest-repeating-character-replacement/

Solution idea: Window length - max frequency <= k.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

36. 3Sum

Topic: Sort + Two Pointers **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/3sum/

Solution idea: Sort; fix one value and solve two-sum with two pointers.

Complexity: Time $O(n^2)$ | Space O(1) extra

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

37. Container With Most Water

Topic: Two Pointers **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/container-with-most-water/

Solution idea: Move the shorter boundary inward because the shorter side limits area.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

38. Two Sum II

Topic: Two Pointers **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/two-sum-ii-input-array-is-sorted/

Solution idea: For sorted array, move left/right based on current sum.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

39. Trapping Rain Water

Topic: Two Pointers **Difficulty:** Hard **Priority:** ★★

Problem link: https://leetcode.com/problems/trapping-rain-water/

Solution idea: Track leftMax/rightMax; process the side with smaller maximum.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

40. Minimum Number of Platforms

Topic: Sorting + Two Pointers **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://www.geeksforgeeks.org/minimum-number-platforms-required-railwaybus-station/

Solution idea: Sort arrivals/departures; if arrival < departure, need another platform, else free one.

Complexity: Time O(n log n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

41. Meeting Rooms

Topic: Sorting **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/meeting-rooms/

Solution idea: Sort intervals by start; overlap exists if current.start < previous.end.

Complexity: Time O(n log n) | Space O(1) extra

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

42. Meeting Rooms II

Topic: Heap **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/meeting-rooms-ii/

Solution idea: Sort by start and use min-heap of end times.

Complexity: Time O(n log n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

43. Non-overlapping Intervals

Topic: Greedy **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/non-overlapping-intervals/

Solution idea: Sort by end; keep interval with earliest ending time.

Complexity: Time O(n log n) | Space O(1) extra

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

44. Interval List Intersections

Topic: Two Pointers **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/interval-list-intersections/

Solution idea: Take max starts/min ends; advance interval with smaller end.

Complexity: Time O(n+m) | Space O(1) extra

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

45. Maximum Number of Overlapping Intervals

Topic: Sweep Line **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://www.geeksforgeeks.org/maximum-number-of-overlapping-intervals/

Solution idea: Represent starts +1 and ends -1, sort events, scan running count.

Complexity: Time O(n log n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

46. Kth Largest Element in an Array

Topic: Heap / Quickselect **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/kth-largest-element-in-an-array/

Solution idea: Use size-k min-heap or quickselect.

Complexity: Time O(n log k) heap | Space O(k)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

47. Min Stack

Topic: Stack **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/min-stack/

Solution idea: Store each value with current minimum, or encode differences.

Complexity: Time O(1) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

48. Next Greater Element I

Topic: Monotonic Stack **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/next-greater-element-i/

Solution idea: Scan nums2 with decreasing stack; map each popped value to current greater value.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
Map<Integer,Integer> m=new HashMap<>();Deque<Integer> st=new ArrayDeque<>();
for(int x:nums2){while(!st.isEmpty()&&st.peek()<x)m.put(st.pop(),x);st.push(x);}
int[] r=new int[nums1.length];for(int i=0;i<r.length;i++)r[i]=m.getOrDefault(nums1[i],-1);return r;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

49. Daily Temperatures

Topic: Monotonic Stack **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/daily-temperatures/

Solution idea: Decreasing stack of indices; current temperature resolves colder previous days.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
int[] r=new int[t.length];Deque<Integer> st=new ArrayDeque<>();
for(int i=0;i<t.length;i++){while(!st.isEmpty()&&t[st.peek()]<t[i]){int j=st.pop();r[j]=i-j;}st.push(i);}
return r;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

50. Evaluate Reverse Polish Notation

Topic: Stack **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/evaluate-reverse-polish-notation/

Solution idea: Push numbers; operators pop b then a and push a op b.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

51. Implement Queue Using Stacks

Topic: Stack **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/implement-queue-using-stacks/

Solution idea: Use input and output stacks; transfer only when output is empty.

Complexity: Time Amortized O(1) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

52. Implement Stack Using Queues

Topic: Queue **Difficulty:** Easy **Priority:** ★★

Problem link: https://leetcode.com/problems/implement-stack-using-queues/

Solution idea: After push, rotate queue so new element becomes front.

Complexity: Time O(n) push | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

53. Largest Rectangle in Histogram

Topic: Monotonic Stack **Difficulty:** Hard **Priority:** ★★★

Problem link: https://leetcode.com/problems/largest-rectangle-in-histogram/

Solution idea: Increasing stack finds previous/next smaller boundaries.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

54. Sliding Window Maximum

Topic: Deque **Difficulty:** Hard **Priority:** ★★★

Problem link: https://leetcode.com/problems/sliding-window-maximum/

Solution idea: Deque stores indices in decreasing value order.

Complexity: Time O(n) | Space O(k)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

55. Decode String

Topic: Stack **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/decode-string/

Solution idea: Stack nested repeat counts and partial strings.

Complexity: Time O(n + output) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

56. Climbing Stairs

Topic: 1D DP **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/climbing-stairs/

Solution idea: dp[i]=dp[i-1]+dp[i-2].

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
int a=1,b=1;
for(int i=0;i<n;i++){int c=a+b;a=b;b=c;} return a;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

57. House Robber

Topic: 1D DP **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/house-robber/

Solution idea: At each house choose rob or skip: max(prev1, prev2+money).

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
int a=0,b=0;
for(int x:nums){int c=Math.max(b,a+x);a=b;b=c;} return b;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

58. Coin Change / Minimum Coins

Topic: DP **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/coin-change/

Solution idea: dp[a]=min(dp[a], dp[a-coin]+1).

Complexity: Time O(amount*coins) | Space O(amount)

Java 21 solution:

```
int[] dp=new int[amount+1];Arrays.fill(dp,amount+1);dp[0]=0;
for(int a=1;a<=amount;a++)for(int c:coins)if(c<=a)dp[a]=Math.min(dp[a],dp[a-c]+1);
return dp[amount]>amount?-1:dp[amount];
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

59. Maximal Square

Topic: 2D DP **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/maximal-square/

Solution idea: For a 1 cell, square size is 1+min(top,left,diagonal).

Complexity: Time O(mn) | Space O(n)

Java 21 solution:

```
int[] dp=new int[matrix[0].length+1];int best=0;
for(int i=1;i<=matrix.length;i++){int diag=0;
for(int j=1;j<=matrix[0].length;j++){int old=dp[j];
if(matrix[i-1][j-1]=='1'){dp[j]=1+Math.min(dp[j],Math.min(dp[j-1],diag));best=Math.max(best,dp[j]);}
else dp[j]=0; diag=old;}}
return best*best;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

60. Unique Paths

Topic: 2D DP **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/unique-paths/

Solution idea: dp[j] += dp[j-1] while traversing grid.

Complexity: Time O(mn) | Space O(n)

Java 21 solution:

```
int[] dp=new int[n];Arrays.fill(dp,1);
for(int i=1;i<m;i++)for(int j=1;j<n;j++)dp[j]+=dp[j-1];
return dp[n-1];
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

61. Minimum Path Sum

Topic: 2D DP **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/minimum-path-sum/

Solution idea: dp cell = value + min(top,left).

Complexity: Time O(mn) | Space O(n)

Java 21 solution:

```
for(int i=0;i<g.length;i++)for(int j=0;j<g[0].length;j++){
if(i==0&&j==0) continue;
int best=Integer.MAX_VALUE;if(i>0)best=g[i-1][j];if(j>0)best=Math.min(best,g[i][j-1]);
g[i][j]+=best;} return g[g.length-1][g[0].length-1];
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

62. Longest Increasing Subsequence

Topic: DP / Binary Search **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/longest-increasing-subsequence/

Solution idea: Maintain smallest possible tail for every subsequence length.

Complexity: Time O(n log n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

63. Longest Common Subsequence

Topic: 2D DP **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/longest-common-subsequence/

Solution idea: If chars equal, diagonal+1; else max(top,left).

Complexity: Time O(mn) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

64. 0/1 Knapsack

Topic: DP **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://www.geeksforgeeks.org/0-1-knapsack-problem-dp-10/

Solution idea: For each item, choose skip or take once; iterate capacity backward in 1D DP.

Complexity: Time O(nW) | Space O(W)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

65. Partition Equal Subset Sum

Topic: 0/1 DP **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/partition-equal-subset-sum/

Solution idea: Target is total/2; boolean subset-sum with backward capacity iteration.

Complexity: Time O(n*sum) | Space O(sum)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

66. Number of Islands

Topic: DFS/BFS **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/number-of-islands/

Solution idea: When finding land, DFS/BFS the entire component and mark visited.

Complexity: Time O(mn) | Space O(mn) worst

Java 21 solution:

```
int ans=0;
for(int i=0;i<g.length;i++)for(int j=0;j<g[0].length;j++)
if(g[i][j]=='1'){ans++;dfs(g,i,j);}
return ans;
// dfs: bounds/visited check, mark '0', recurse 4 directions.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

67. Rotting Oranges

Topic: Multi-source BFS **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/rotting-oranges/

Solution idea: Start BFS with all rotten cells at once; each layer is one minute.

Complexity: Time O(mn) | Space O(mn)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

68. Flood Fill

Topic: DFS/BFS **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/flood-fill/

Solution idea: DFS/BFS from source and recolor connected cells of original color.

Complexity: Time O(mn) | Space O(mn)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

69. Spiral Matrix

Topic: Matrix traversal **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/spiral-matrix/

Solution idea: Maintain top,bottom,left,right boundaries.

Complexity: Time O(mn) | Space O(1) extra

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

70. Rotate Image

Topic: Matrix **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/rotate-image/

Solution idea: Transpose matrix then reverse each row.

Complexity: Time O(n²) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

71. Search a 2D Matrix

Topic: Binary Search **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/search-a-2d-matrix/

Solution idea: Treat sorted matrix as a flattened sorted array.

Complexity: Time O(log mn) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

72. Set Matrix Zeroes

Topic: Matrix / In-place **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/set-matrix-zeroes/

Solution idea: Use first row/column as markers; preserve their original zero status.

Complexity: Time O(mn) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

73. Word Search

Topic: Backtracking **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/word-search/

Solution idea: DFS four directions, mark current cell temporarily, then restore.

Complexity: Time O(mn*4^L) | Space O(L) recursion

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

74. Reverse Linked List

Topic: Linked List **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/reverse-linked-list/

Solution idea: Iteratively reverse next pointers using prev/current.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
ListNode prev=null,cur=head;
while(cur!=null){ListNode next=cur.next;cur.next=prev;prev=cur;cur=next;}
return prev;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

75. Merge Two Sorted Lists

Topic: Linked List **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/merge-two-sorted-lists/

Solution idea: Use dummy node and attach smaller current node each step.

Complexity: Time O(n+m) | Space O(1)

Java 21 solution:

```
ListNode d=new ListNode(0),p=d;
while(a!=null&&b!=null){if(a.val<=b.val){p.next=a;a=a.next;}else{p.next=b;b=b.next;}p=p.next;}
p.next=(a!=null?a:b);return d.next;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

76. Linked List Cycle

Topic: Fast/Slow **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://leetcode.com/problems/linked-list-cycle/

Solution idea: Floyd's slow/fast pointers meet iff a cycle exists.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
ListNode slow=head,fast=head;
while(fast!=null&&fast.next!=null){slow=slow.next;fast=fast.next.next;if(slow==fast)return true;}
return false;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

77. Remove Nth Node From End

Topic: Two Pointers **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/remove-nth-node-from-end-of-list/

Solution idea: Move fast n+1 ahead, then move both; delete fast.next when fast reaches end.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

78. Middle of Linked List

Topic: Fast/Slow **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/middle-of-the-linked-list/

Solution idea: Slow moves one step, fast two; slow ends at middle.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
ListNode slow=head,fast=head;
while(fast!=null&&fast.next!=null){slow=slow.next;fast=fast.next.next;}return slow;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

79. Kth Largest Element

Topic: Heap **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/kth-largest-element-in-an-array/

Solution idea: Maintain a min-heap of k largest values.

Complexity: Time O(n log k) | Space O(k)

Java 21 solution:

```
PriorityQueue<Integer> pq=new PriorityQueue<>();
for(int x:nums){pq.offer(x);if(pq.size()>k)pq.poll();}return pq.peek();
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

80. Top K Frequent Elements

Topic: Heap + HashMap **Difficulty:** Medium **Priority:** ★★★★★

Problem link: https://leetcode.com/problems/top-k-frequent-elements/

Solution idea: Count then keep k highest frequencies in min-heap.

Complexity: Time O(n log k) | Space O(n)

Java 21 solution:

```
Map<Integer,Integer> f=new HashMap<>();
for(int x:nums) f.merge(x,1,Integer::sum);
PriorityQueue<int[]> pq=new PriorityQueue<>(Comparator.comparingInt(a->a[1]));
for(var e:f.entrySet()){pq.offer(new int[]{e.getKey(),e.getValue()});if(pq.size()>k)pq.poll();}
int[] r=new int[k];for(int i=0;i<k;i++)r[i]=pq.poll()[0];return r;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

81. Merge K Sorted Lists

Topic: Heap **Difficulty:** Hard **Priority:** ★★★

Problem link: https://leetcode.com/problems/merge-k-sorted-lists/

Solution idea: Push each list head into min-heap and repeatedly extract minimum.

Complexity: Time O(N log k) | Space O(k)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

82. K Closest Points to Origin

Topic: Heap **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/k-closest-points-to-origin/

Solution idea: Max-heap of size k keyed by squared distance.

Complexity: Time O(n log k) | Space O(k)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

83. Minimum Operations to Make Array Unique ■

Topic: Heap / Greedy **Difficulty:** Medium **Priority:** ★★★★★

Problem link: Custom/reported pattern — no single canonical problem page.

Solution idea: JPMC-reported style. Sort values or use a min-heap; repeatedly raise duplicates to the next free integer and accumulate increments. Exact constraints vary by report.

Complexity: Time O(n log n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

84. Binary Tree Level Order Traversal

Topic: BFS **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/binary-tree-level-order-traversal/

Solution idea: Queue nodes level by level.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
List<List<Integer>> out=new ArrayList<>();if(root==null)return out;
Deque<TreeNode> q=new ArrayDeque<>();q.add(root);
while(!q.isEmpty()){int n=q.size();List<Integer> row=new ArrayList<>();
while(n-->0){TreeNode x=q.poll();row.add(x.val);if(x.left!=null)q.add(x.left);if(x.right!=null)q.add(x.right);}
out.add(row);}return out;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

85. Maximum Depth of Binary Tree

Topic: DFS **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/maximum-depth-of-binary-tree/

Solution idea: Depth = 1 + max(leftDepth,rightDepth).

Complexity: Time O(n) | Space O(h)

Java 21 solution:

```
return root==null?0:1+Math.max(maxDepth(root.left),maxDepth(root.right));
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

86. Validate Binary Search Tree

Topic: DFS / Bounds **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/validate-binary-search-tree/

Solution idea: Every node must lie within valid lower/upper bounds.

Complexity: Time O(n) | Space O(h)

Java 21 solution:

```
boolean dfs(TreeNode n,long lo,long hi){
if(n==null)return true;if(n.val<=lo||n.val>=hi)return false;
return dfs(n.left,lo,n.val)&&dfs(n.right,n.val,hi);
} // call dfs(root,Long.MIN_VALUE,Long.MAX_VALUE)
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

87. Number of Connected Components

Topic: Graph DFS/Union-Find **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph/

Solution idea: DFS each unvisited node or use DSU to count components.

Complexity: Time O(V+E) | Space O(V)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

88. Shortest Path / Dijkstra

Topic: Graph / Heap **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/network-delay-time/

Solution idea: Dijkstra: repeatedly finalize the unvisited node with smallest distance using a min-heap.

Complexity: Time O((V+E)log V) | Space O(V+E)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

89. Reverse and Add Until Palindrome ■

Topic: Math / Simulation **Difficulty:** Easy **Priority:** ★★★★

Problem link: https://www.geeksforgeeks.org/reverse-and-add-function/

Solution idea: Reverse the integer, add it, and repeat until palindrome; guard against overflow/iteration constraints.

Complexity: Time Depends | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

90. JPMC 2D Level-Connection Counting Pattern ■

Topic: Matrix / Counting **Difficulty:** Medium **Priority:** ★★★★

Problem link: Custom/reported pattern — no single canonical problem page.

Solution idea: Reported JPMC custom pattern: process successive non-empty levels/rows and count valid connections according to the stated rule. Exact formulation is interviewer-specific, so clarify the connection rule before coding.

Complexity: Time O(mn) typical | Space O(1) extra

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

91. Maximum Product Subarray

Topic: DP / Array **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/maximum-product-subarray/

Solution idea: Track both maximum and minimum product ending at the current position because a negative can swap them.

Complexity: Time O(n) | Space O(1)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

92. Reverse Words in a String

Topic: String **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/reverse-words-in-a-string/

Solution idea: Read words from right to left or split, reverse and join.

Complexity: Time O(n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

93. Longest Increasing Subsequence

Topic: DP / Binary Search **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/longest-increasing-subsequence/

Solution idea: Maintain the smallest possible tail for each subsequence length; use lower_bound logic.

Complexity: Time O(n log n) | Space O(n)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

94. Partition Equal Subset Sum

Topic: 0/1 DP **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/partition-equal-subset-sum/

Solution idea: Target is total/2; run boolean subset-sum with capacity backwards.

Complexity: Time O(n*sum) | Space O(sum)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

95. Spiral Matrix

Topic: Matrix **Difficulty:** Medium **Priority:** ★★★★

Problem link: https://leetcode.com/problems/spiral-matrix/

Solution idea: Maintain top, bottom, left and right boundaries and peel layers.

Complexity: Time O(mn) | Space O(1) extra

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

96. Word Search

Topic: Backtracking **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/word-search/

Solution idea: DFS four directions, mark the current cell, recurse, then restore it.

Complexity: Time O(mn*4^L) | Space O(L) recursion

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

97. Merge Two Sorted Lists

Topic: Linked List **Difficulty:** Easy **Priority:** ★★★

Problem link: https://leetcode.com/problems/merge-two-sorted-lists/

Solution idea: Use a dummy node and repeatedly attach the smaller head.

Complexity: Time O(n+m) | Space O(1)

Java 21 solution:

```
ListNode d=new ListNode(0),p=d;
while(a!=null&&b!=null){if(a.val<=b.val){p.next=a;a=a.next;}else{p.next=b;b=b.next;}p=p.next;}
p.next=(a!=null?a:b);return d.next;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

98. Merge K Sorted Lists

Topic: Heap **Difficulty:** Hard **Priority:** ★★★

Problem link: https://leetcode.com/problems/merge-k-sorted-lists/

Solution idea: Put each non-null list head into a min-heap and repeatedly extract the smallest.

Complexity: Time O(N log k) | Space O(k)

Java 21 solution:

```
// Java 21 implementation template:
// Apply the approach above using the stated data structure/algorithm.
// Verify constraints, edge cases and overflow before final submission.
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

99. Validate Binary Search Tree

Topic: Tree / DFS **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/validate-binary-search-tree/

Solution idea: DFS with lower/upper bounds; every node must stay inside its valid range.

Complexity: Time O(n) | Space O(h)

Java 21 solution:

```
boolean dfs(TreeNode n,long lo,long hi){
if(n==null)return true;if(n.val<=lo||n.val>=hi)return false;
return dfs(n.left,lo,n.val)&&dfs(n.right,n.val,hi);
} // call dfs(root,Long.MIN_VALUE,Long.MAX_VALUE)
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

100. Course Schedule

Topic: Graph / Topological Sort **Difficulty:** Medium **Priority:** ★★★

Problem link: https://leetcode.com/problems/course-schedule/

Solution idea: Build prerequisite graph and use Kahn's BFS indegree algorithm; a cycle means impossible.

Complexity: Time O(V+E) | Space O(V+E)

Java 21 solution:

```
int[] in=new int[numCourses];List<Integer>[] g=new ArrayList[numCourses];
for(int i=0;i<numCourses;i++)g[i]=new ArrayList<>();
for(int[] e:prerequisites){g[e[1]].add(e[0]);in[e[0]]++;}
Deque<Integer> q=new ArrayDeque<>();for(int i=0;i<numCourses;i++)if(in[i]==0)q.add(i);
int seen=0;while(!q.isEmpty()){int u=q.poll();seen++;for(int v:g[u])if(--in[v]==0)q.add(v);}
return seen==numCourses;
```

Interview follow-up: Be ready to explain brute force, why the optimized approach works, complexity, edge cases, and how the solution changes if constraints are larger.

JPMC Final Revision Checklist

Master first: Two Sum, Valid Parentheses, Longest Substring Without Repeating Characters, Subarray Sum Equals K, Merge Intervals, Minimum Platforms, Meeting Rooms II, Coin Change, House Robber, Maximal Square, Number of Islands, Top K Frequent, Kth Largest, Next Greater Element, Daily Temperatures.

Java 21: HashMap, HashSet, ArrayDeque, PriorityQueue, Arrays.sort, Comparator, StringBuilder, recursion, BFS/DFS, and clean class/method structure.

Important: A reported JPMC question is evidence of a candidate experience, not a guarantee that the same question will appear in your interview.